

Figure 32.9 An illustration for the proof of Lemma 32.3. The figure shows that $r = \sigma(P_q a)$, where $q = \sigma(x)$ and $r = \sigma(xa)$.

dropping the a from the end of P_r and from the end of xa . Therefore, $r - 1 \leq \sigma(x)$, since $\sigma(x)$ is the largest k such that $P_k \sqsubset x$, and thus $\sigma(xa) = r \leq \sigma(x) + 1$. ■

Lemma 32.3 (Suffix-function recursion lemma)

For any string x and character a , if $q = \sigma(x)$, then $\sigma(xa) = \sigma(P_q a)$.

Proof From the definition of σ , we have $P_q \sqsubset x$. As Figure 32.9 shows, we also have $P_q a \sqsubset xa$. If we let $r = \sigma(xa)$, then $P_r \sqsubset xa$ and, by Lemma 32.2, $r \leq q + 1$. Thus, we have $|P_r| = r \leq q + 1 = |P_q a|$. Since $P_q a \sqsubset xa$, $P_r \sqsubset xa$, and $|P_r| \leq |P_q a|$, Lemma 32.1 implies that $P_r \sqsubset P_q a$. Therefore, $r \leq \sigma(P_q a)$, that is, $\sigma(xa) \leq \sigma(P_q a)$. But we also have $\sigma(P_q a) \leq \sigma(xa)$, since $P_q a \sqsubset xa$. Thus, $\sigma(xa) = \sigma(P_q a)$. ■

We are now ready to prove our main theorem characterizing the behavior of a string-matching automaton on a given input text. As noted above, this theorem shows that the automaton is merely keeping track, at each step, of the longest prefix of the pattern that is a suffix of what has been read so far. In other words, the automaton maintains the invariant (32.5).

Theorem 32.4

If ϕ is the final-state function of a string-matching automaton for a given pattern P and $T[1..n]$ is an input text for the automaton, then

$$\phi(T_i) = \sigma(T_i)$$

for $i = 0, 1, \dots, n$.

Proof The proof is by induction on i . For $i = 0$, the theorem is trivially true, since $T_0 = \varepsilon$. Thus, $\phi(T_0) = 0 = \sigma(T_0)$.

Now, we assume that $\phi(T_i) = \sigma(T_i)$ and prove that $\phi(T_{i+1}) = \sigma(T_{i+1})$. Let q denote $\phi(T_i)$, and let a denote $T[i + 1]$. Then,

$$\begin{aligned}
 \phi(T_{i+1}) &= \phi(T_i a) && \text{(by the definitions of } T_{i+1} \text{ and } a) \\
 &= \delta(\phi(T_i), a) && \text{(by the definition of } \phi) \\
 &= \delta(q, a) && \text{(by the definition of } q) \\
 &= \sigma(P_q a) && \text{(by the definition (32.4) of } \delta) \\
 &= \sigma(T_i a) && \text{(by Lemma 32.3 and induction)} \\
 &= \sigma(T_{i+1}) && \text{(by the definition of } T_{i+1}) \quad \blacksquare
 \end{aligned}$$

By Theorem 32.4, if the machine enters state q on line 4, then q is the largest value such that $P_q \sqsupset T_i$. Thus, we have $q = m$ on line 5 if and only if the machine has just scanned an occurrence of the pattern P . We conclude that FINITE-AUTOMATON-MATCHER operates correctly.

Computing the transition function

The following procedure computes the transition function δ from a given pattern $P[1..m]$.

COMPUTE-TRANSITION-FUNCTION(P, Σ)

```

1   $m = P.length$ 
2  for  $q = 0$  to  $m$ 
3      for each character  $a \in \Sigma$ 
4           $k = \min(m + 1, q + 2)$ 
5          repeat
6               $k = k - 1$ 
7          until  $P_k \sqsupset P_q a$ 
8               $\delta(q, a) = k$ 
9  return  $\delta$ 

```

This procedure computes $\delta(q, a)$ in a straightforward manner according to its definition in equation (32.4). The nested loops beginning on lines 2 and 3 consider all states q and all characters a , and lines 4–8 set $\delta(q, a)$ to be the largest k such that $P_k \sqsupset P_q a$. The code starts with the largest conceivable value of k , which is $\min(m, q + 1)$. It then decreases k until $P_k \sqsupset P_q a$, which must eventually occur, since $P_0 = \varepsilon$ is a suffix of every string.

The running time of COMPUTE-TRANSITION-FUNCTION is $O(m^3 |\Sigma|)$, because the outer loops contribute a factor of $m |\Sigma|$, the inner **repeat** loop can run at most $m + 1$ times, and the test $P_k \sqsupset P_q a$ on line 7 can require comparing up

to m characters. Much faster procedures exist; by utilizing some cleverly computed information about the pattern P (see Exercise 32.4-8), we can improve the time required to compute δ from P to $O(m |\Sigma|)$. With this improved procedure for computing δ , we can find all occurrences of a length- m pattern in a length- n text over an alphabet Σ with $O(m |\Sigma|)$ preprocessing time and $\Theta(n)$ matching time.

Exercises

32.3-1

Construct the string-matching automaton for the pattern $P = \text{aabab}$ and illustrate its operation on the text string $T = \text{aaababaabaababaab}$.

32.3-2

Draw a state-transition diagram for a string-matching automaton for the pattern $\text{ababbabbababbababbabb}$ over the alphabet $\Sigma = \{\text{a}, \text{b}\}$.

32.3-3

We call a pattern P **nonoverlappable** if $P_k \sqsubset P_q$ implies $k = 0$ or $k = q$. Describe the state-transition diagram of the string-matching automaton for a nonoverlappable pattern.

32.3-4 ★

Given two patterns P and P' , describe how to construct a finite automaton that determines all occurrences of *either* pattern. Try to minimize the number of states in your automaton.

32.3-5

Given a pattern P containing gap characters (see Exercise 32.1-4), show how to build a finite automaton that can find an occurrence of P in a text T in $O(n)$ matching time, where $n = |T|$.

★ 32.4 The Knuth-Morris-Pratt algorithm

We now present a linear-time string-matching algorithm due to Knuth, Morris, and Pratt. This algorithm avoids computing the transition function δ altogether, and its matching time is $\Theta(n)$ using just an auxiliary function π , which we precompute from the pattern in time $\Theta(m)$ and store in an array $\pi[1..m]$. The array π allows us to compute the transition function δ efficiently (in an amortized sense) “on the fly” as needed. Loosely speaking, for any state $q = 0, 1, \dots, m$ and any character

$a \in \Sigma$, the value $\pi[q]$ contains the information we need to compute $\delta(q, a)$ but that does not depend on a . Since the array π has only m entries, whereas δ has $\Theta(m |\Sigma|)$ entries, we save a factor of $|\Sigma|$ in the preprocessing time by computing π rather than δ .

The prefix function for a pattern

The prefix function π for a pattern encapsulates knowledge about how the pattern matches against shifts of itself. We can take advantage of this information to avoid testing useless shifts in the naive pattern-matching algorithm and to avoid precomputing the full transition function δ for a string-matching automaton.

Consider the operation of the naive string matcher. Figure 32.10(a) shows a particular shift s of a template containing the pattern $P = \text{ababaca}$ against a text T . For this example, $q = 5$ of the characters have matched successfully, but the 6th pattern character fails to match the corresponding text character. The information that q characters have matched successfully determines the corresponding text characters. Knowing these q text characters allows us to determine immediately that certain shifts are invalid. In the example of the figure, the shift $s + 1$ is necessarily invalid, since the first pattern character (a) would be aligned with a text character that we know does not match the first pattern character, but does match the second pattern character (b). The shift $s' = s + 2$ shown in part (b) of the figure, however, aligns the first three pattern characters with three text characters that must necessarily match. In general, it is useful to know the answer to the following question:

Given that pattern characters $P[1..q]$ match text characters $T[s+1..s+q]$, what is the least shift $s' > s$ such that for some $k < q$,

$$P[1..k] = T[s' + 1..s' + k], \quad (32.6)$$

where $s' + k = s + q$?

In other words, knowing that $P_q \sqsupseteq T_{s+q}$, we want the longest proper prefix P_k of P_q that is also a suffix of T_{s+q} . (Since $s' + k = s + q$, if we are given s and q , then finding the smallest shift s' is tantamount to finding the longest prefix length k .) We add the difference $q - k$ in the lengths of these prefixes of P to the shift s to arrive at our new shift s' , so that $s' = s + (q - k)$. In the best case, $k = 0$, so that $s' = s + q$, and we immediately rule out shifts $s + 1, s + 2, \dots, s + q - 1$. In any case, at the new shift s' we don't need to compare the first k characters of P with the corresponding characters of T , since equation (32.6) guarantees that they match.

We can precompute the necessary information by comparing the pattern against itself, as Figure 32.10(c) demonstrates. Since $T[s' + 1..s' + k]$ is part of the

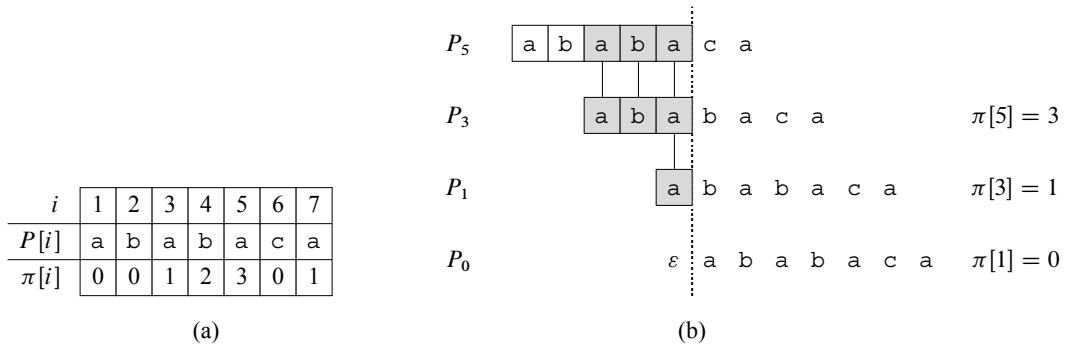


Figure 32.11 An illustration of Lemma 32.5 for the pattern $P = ababaca$ and $q = 5$. **(a)** The π function for the given pattern. Since $\pi[5] = 3$, $\pi[3] = 1$, and $\pi[1] = 0$, by iterating π we obtain $\pi^*[5] = \{3, 1, 0\}$. **(b)** We slide the template containing the pattern P to the right and note when some prefix P_k of P matches up with some proper suffix of P_5 ; we get matches when $k = 3, 1$, and 0 . In the figure, the first row gives P , and the dotted vertical line is drawn just after P_5 . Successive rows show all the shifts of P that cause some prefix P_k of P to match some suffix of P_5 . Successfully matched characters are shown shaded. Vertical lines connect aligned matching characters. Thus, $\{k : k < 5 \text{ and } P_k \sqsubset P_5\} = \{3, 1, 0\}$. Lemma 32.5 claims that $\pi^*[q] = \{k : k < q \text{ and } P_k \sqsubset P_q\}$ for all q .

The pseudocode below gives the Knuth-Morris-Pratt matching algorithm as the procedure KMP-MATCHER. For the most part, the procedure follows from FINITE-AUTOMATON-MATCHER, as we shall see. KMP-MATCHER calls the auxiliary procedure COMPUTE-PREFIX-FUNCTION to compute π .

KMP-MATCHER(T, P)

```

1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

COMPUTE-PREFIX-FUNCTION(P)

```

1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 

```

These two procedures have much in common, because both match a string against the pattern P : KMP-MATCHER matches the text T against P , and COMPUTE-PREFIX-FUNCTION matches P against itself.

We begin with an analysis of the running times of these procedures. Proving these procedures correct will be more complicated.

Running-time analysis

The running time of COMPUTE-PREFIX-FUNCTION is $\Theta(m)$, which we show by using the aggregate method of amortized analysis (see Section 17.1). The only tricky part is showing that the **while** loop of lines 6–7 executes $O(m)$ times altogether. We shall show that it makes at most $m - 1$ iterations. We start by making some observations about k . First, line 4 starts k at 0, and the only way that k increases is by the increment operation in line 9, which executes at most once per iteration of the **for** loop of lines 5–10. Thus, the total increase in k is at most $m - 1$. Second, since $k < q$ upon entering the **for** loop and each iteration of the loop increments q , we always have $k < q$. Therefore, the assignments in lines 3 and 10 ensure that $\pi[q] < q$ for all $q = 1, 2, \dots, m$, which means that each iteration of the **while** loop decreases k . Third, k never becomes negative. Putting these facts together, we see that the total decrease in k from the **while** loop is bounded from above by the total increase in k over all iterations of the **for** loop, which is $m - 1$. Thus, the **while** loop iterates at most $m - 1$ times in all, and COMPUTE-PREFIX-FUNCTION runs in time $\Theta(m)$.

Exercise 32.4-4 asks you to show, by a similar aggregate analysis, that the matching time of KMP-MATCHER is $\Theta(n)$.

Compared with FINITE-AUTOMATON-MATCHER, by using π rather than δ , we have reduced the time for preprocessing the pattern from $O(m |\Sigma|)$ to $\Theta(m)$, while keeping the actual matching time bounded by $\Theta(n)$.

Correctness of the prefix-function computation

We shall see a little later that the prefix function π helps us simulate the transition function δ in a string-matching automaton. But first, we need to prove that the procedure COMPUTE-PREFIX-FUNCTION does indeed compute the prefix function correctly. In order to do so, we will need to find all prefixes P_k that are proper suffixes of a given prefix P_q . The value of $\pi[q]$ gives us the longest such prefix, but the following lemma, illustrated in Figure 32.11, shows that by iterating the prefix function π , we can indeed enumerate all the prefixes P_k that are proper suffixes of P_q . Let

$$\pi^*[q] = \{\pi[q], \pi^{(2)}[q], \pi^{(3)}[q], \dots, \pi^{(t)}[q]\},$$

where $\pi^{(i)}[q]$ is defined in terms of functional iteration, so that $\pi^{(0)}[q] = q$ and $\pi^{(i)}[q] = \pi[\pi^{(i-1)}[q]]$ for $i \geq 1$, and where the sequence in $\pi^*[q]$ stops upon reaching $\pi^{(t)}[q] = 0$.

Lemma 32.5 (Prefix-function iteration lemma)

Let P be a pattern of length m with prefix function π . Then, for $q = 1, 2, \dots, m$, we have $\pi^*[q] = \{k : k < q \text{ and } P_k \sqsupset P_q\}$.

Proof We first prove that $\pi^*[q] \subseteq \{k : k < q \text{ and } P_k \sqsupset P_q\}$ or, equivalently,

$$i \in \pi^*[q] \text{ implies } P_i \sqsupset P_q. \quad (32.7)$$

If $i \in \pi^*[q]$, then $i = \pi^{(u)}[q]$ for some $u > 0$. We prove equation (32.7) by induction on u . For $u = 1$, we have $i = \pi[q]$, and the claim follows since $i < q$ and $P_{\pi[q]} \sqsupset P_q$ by the definition of π . Using the relations $\pi[i] < i$ and $P_{\pi[i]} \sqsupset P_i$ and the transitivity of $<$ and $\sqsupset$ establishes the claim for all i in $\pi^*[q]$. Therefore, $\pi^*[q] \subseteq \{k : k < q \text{ and } P_k \sqsupset P_q\}$.

We now prove that $\{k : k < q \text{ and } P_k \sqsupset P_q\} \subseteq \pi^*[q]$ by contradiction. Suppose to the contrary that the set $\{k : k < q \text{ and } P_k \sqsupset P_q\} - \pi^*[q]$ is nonempty, and let j be the largest number in the set. Because $\pi[q]$ is the largest value in $\{k : k < q \text{ and } P_k \sqsupset P_q\}$ and $\pi[q] \in \pi^*[q]$, we must have $j < \pi[q]$, and so we let j' denote the smallest integer in $\pi^*[q]$ that is greater than j . (We can choose $j' = \pi[q]$ if no other number in $\pi^*[q]$ is greater than j .) We have $P_j \sqsupset P_q$ because $j \in \{k : k < q \text{ and } P_k \sqsupset P_q\}$, and from $j' \in \pi^*[q]$ and equation (32.7), we have $P_{j'} \sqsupset P_q$. Thus, $P_j \sqsupset P_{j'}$ by Lemma 32.1, and j is the largest value less than j' with this property. Therefore, we must have $\pi[j'] = j$ and, since $j' \in \pi^*[q]$, we must have $j \in \pi^*[q]$ as well. This contradiction proves the lemma. ■

The algorithm COMPUTE-PREFIX-FUNCTION computes $\pi[q]$, in order, for $q = 1, 2, \dots, m$. Setting $\pi[1]$ to 0 in line 3 of COMPUTE-PREFIX-FUNCTION is certainly correct, since $\pi[q] < q$ for all q . We shall use the following lemma and

its corollary to prove that COMPUTE-PREFIX-FUNCTION computes $\pi[q]$ correctly for $q > 1$.

Lemma 32.6

Let P be a pattern of length m , and let π be the prefix function for P . For $q = 1, 2, \dots, m$, if $\pi[q] > 0$, then $\pi[q] - 1 \in \pi^*[q - 1]$.

Proof Let $r = \pi[q] > 0$, so that $r < q$ and $P_r \sqsubset P_q$; thus, $r - 1 < q - 1$ and $P_{r-1} \sqsubset P_{q-1}$ (by dropping the last character from P_r and P_q , which we can do because $r > 0$). By Lemma 32.5, therefore, $r - 1 \in \pi^*[q - 1]$. Thus, we have $\pi[q] - 1 = r - 1 \in \pi^*[q - 1]$. ■

For $q = 2, 3, \dots, m$, define the subset $E_{q-1} \subseteq \pi^*[q - 1]$ by

$$\begin{aligned} E_{q-1} &= \{k \in \pi^*[q - 1] : P[k + 1] = P[q]\} \\ &= \{k : k < q - 1 \text{ and } P_k \sqsubset P_{q-1} \text{ and } P[k + 1] = P[q]\} \text{ (by Lemma 32.5)} \\ &= \{k : k < q - 1 \text{ and } P_{k+1} \sqsubset P_q\} . \end{aligned}$$

The set E_{q-1} consists of the values $k < q - 1$ for which $P_k \sqsubset P_{q-1}$ and for which, because $P[k + 1] = P[q]$, we have $P_{k+1} \sqsubset P_q$. Thus, E_{q-1} consists of those values $k \in \pi^*[q - 1]$ such that we can extend P_k to P_{k+1} and get a proper suffix of P_q .

Corollary 32.7

Let P be a pattern of length m , and let π be the prefix function for P . For $q = 2, 3, \dots, m$,

$$\pi[q] = \begin{cases} 0 & \text{if } E_{q-1} = \emptyset , \\ 1 + \max \{k \in E_{q-1}\} & \text{if } E_{q-1} \neq \emptyset . \end{cases}$$

Proof If E_{q-1} is empty, there is no $k \in \pi^*[q - 1]$ (including $k = 0$) for which we can extend P_k to P_{k+1} and get a proper suffix of P_q . Therefore $\pi[q] = 0$.

If E_{q-1} is nonempty, then for each $k \in E_{q-1}$ we have $k + 1 < q$ and $P_{k+1} \sqsubset P_q$. Therefore, from the definition of $\pi[q]$, we have

$$\pi[q] \geq 1 + \max \{k \in E_{q-1}\} . \quad (32.8)$$

Note that $\pi[q] > 0$. Let $r = \pi[q] - 1$, so that $r + 1 = \pi[q]$ and therefore $P_{r+1} \sqsubset P_q$. Since $r + 1 > 0$, we have $P[r + 1] = P[q]$. Furthermore, by Lemma 32.6, we have $r \in \pi^*[q - 1]$. Therefore, $r \in E_{q-1}$, and so $r \leq \max \{k \in E_{q-1}\}$ or, equivalently,

$$\pi[q] \leq 1 + \max \{k \in E_{q-1}\} . \quad (32.9)$$

Combining equations (32.8) and (32.9) completes the proof. ■

We now finish the proof that COMPUTE-PREFIX-FUNCTION computes π correctly. In the procedure COMPUTE-PREFIX-FUNCTION, at the start of each iteration of the **for** loop of lines 5–10, we have that $k = \pi[q - 1]$. This condition is enforced by lines 3 and 4 when the loop is first entered, and it remains true in each successive iteration because of line 10. Lines 6–9 adjust k so that it becomes the correct value of $\pi[q]$. The **while** loop of lines 6–7 searches through all values $k \in \pi^*[q - 1]$ until it finds a value of k for which $P[k + 1] = P[q]$; at that point, k is the largest value in the set E_{q-1} , so that, by Corollary 32.7, we can set $\pi[q]$ to $k + 1$. If the **while** loop cannot find a $k \in \pi^*[q - 1]$ such that $P[k + 1] = P[q]$, then k equals 0 at line 8. If $P[1] = P[q]$, then we should set both k and $\pi[q]$ to 1; otherwise we should leave k alone and set $\pi[q]$ to 0. Lines 8–10 set k and $\pi[q]$ correctly in either case. This completes our proof of the correctness of COMPUTE-PREFIX-FUNCTION.

Correctness of the Knuth-Morris-Pratt algorithm

We can think of the procedure KMP-MATCHER as a reimplemented version of the procedure FINITE-AUTOMATON-MATCHER, but using the prefix function π to compute state transitions. Specifically, we shall prove that in the i th iteration of the **for** loops of both KMP-MATCHER and FINITE-AUTOMATON-MATCHER, the state q has the same value when we test for equality with m (at line 10 in KMP-MATCHER and at line 5 in FINITE-AUTOMATON-MATCHER). Once we have argued that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER, the correctness of KMP-MATCHER follows from the correctness of FINITE-AUTOMATON-MATCHER (though we shall see a little later why line 12 in KMP-MATCHER is necessary).

Before we formally prove that KMP-MATCHER correctly simulates FINITE-AUTOMATON-MATCHER, let's take a moment to understand how the prefix function π replaces the δ transition function. Recall that when a string-matching automaton is in state q and it scans a character $a = T[i]$, it moves to a new state $\delta(q, a)$. If $a = P[q + 1]$, so that a continues to match the pattern, then $\delta(q, a) = q + 1$. Otherwise, $a \neq P[q + 1]$, so that a does not continue to match the pattern, and $0 \leq \delta(q, a) \leq q$. In the first case, when a continues to match, KMP-MATCHER moves to state $q + 1$ without referring to the π function: the **while** loop test in line 6 comes up false the first time, the test in line 8 comes up true, and line 9 increments q .

The π function comes into play when the character a does not continue to match the pattern, so that the new state $\delta(q, a)$ is either q or to the left of q along the spine of the automaton. The **while** loop of lines 6–7 in KMP-MATCHER iterates through the states in $\pi^*[q]$, stopping either when it arrives in a state, say q' , such that a matches $P[q' + 1]$ or q' has gone all the way down to 0. If a matches $P[q' + 1]$,

then line 9 sets the new state to $q' + 1$, which should equal $\delta(q, a)$ for the simulation to work correctly. In other words, the new state $\delta(q, a)$ should be either state 0 or one greater than some state in $\pi^*[q]$.

Let's look at the example in Figures 32.7 and 32.11, which are for the pattern $P = \text{ababaca}$. Suppose that the automaton is in state $q = 5$; the states in $\pi^*[5]$ are, in descending order, 3, 1, and 0. If the next character scanned is c , then we can easily see that the automaton moves to state $\delta(5, c) = 6$ in both **FINITE-AUTOMATON-MATCHER** and **KMP-MATCHER**. Now suppose that the next character scanned is instead b , so that the automaton should move to state $\delta(5, b) = 4$. The **while** loop in **KMP-MATCHER** exits having executed line 7 once, and it arrives in state $q' = \pi[5] = 3$. Since $P[q' + 1] = P[4] = b$, the test in line 8 comes up true, and **KMP-MATCHER** moves to the new state $q' + 1 = 4 = \delta(5, b)$. Finally, suppose that the next character scanned is instead a , so that the automaton should move to state $\delta(5, a) = 1$. The first three times that the test in line 6 executes, the test comes up true. The first time, we find that $P[6] = c \neq a$, and **KMP-MATCHER** moves to state $\pi[5] = 3$ (the first state in $\pi^*[5]$). The second time, we find that $P[4] = b \neq a$ and move to state $\pi[3] = 1$ (the second state in $\pi^*[5]$). The third time, we find that $P[2] = b \neq a$ and move to state $\pi[1] = 0$ (the last state in $\pi^*[5]$). The **while** loop exits once it arrives in state $q' = 0$. Now, line 8 finds that $P[q' + 1] = P[1] = a$, and line 9 moves the automaton to the new state $q' + 1 = 1 = \delta(5, a)$.

Thus, our intuition is that **KMP-MATCHER** iterates through the states in $\pi^*[q]$ in decreasing order, stopping at some state q' and then possibly moving to state $q' + 1$. Although that might seem like a lot of work just to simulate computing $\delta(q, a)$, bear in mind that asymptotically, **KMP-MATCHER** is no slower than **FINITE-AUTOMATON-MATCHER**.

We are now ready to formally prove the correctness of the Knuth-Morris-Pratt algorithm. By Theorem 32.4, we have that $q = \sigma(T_i)$ after each time we execute line 4 of **FINITE-AUTOMATON-MATCHER**. Therefore, it suffices to show that the same property holds with regard to the **for** loop in **KMP-MATCHER**. The proof proceeds by induction on the number of loop iterations. Initially, both procedures set q to 0 as they enter their respective **for** loops for the first time. Consider iteration i of the **for** loop in **KMP-MATCHER**, and let q' be state at the start of this loop iteration. By the inductive hypothesis, we have $q' = \sigma(T_{i-1})$. We need to show that $q = \sigma(T_i)$ at line 10. (Again, we shall handle line 12 separately.)

When we consider the character $T[i]$, the longest prefix of P that is a suffix of T_i is either $P_{q'+1}$ (if $P[q' + 1] = T[i]$) or some prefix (not necessarily proper, and possibly empty) of $P_{q'}$. We consider separately the three cases in which $\sigma(T_i) = 0$, $\sigma(T_i) = q' + 1$, and $0 < \sigma(T_i) \leq q'$.

- If $\sigma(T_i) = 0$, then $P_0 = \varepsilon$ is the only prefix of P that is a suffix of T_i . The **while** loop of lines 6–7 iterates through the values in $\pi^*[q']$, but although $P_q \sqsubset T_{i-1}$ for every $q \in \pi^*[q']$, the loop never finds a q such that $P[q+1] = T[i]$. The loop terminates when q reaches 0, and of course line 9 does not execute. Therefore, $q = 0$ at line 10, so that $q = \sigma(T_i)$.
- If $\sigma(T_i) = q' + 1$, then $P[q' + 1] = T[i]$, and the **while** loop test in line 6 fails the first time through. Line 9 executes, incrementing q so that afterward we have $q = q' + 1 = \sigma(T_i)$.
- If $0 < \sigma(T_i) \leq q'$, then the **while** loop of lines 6–7 iterates at least once, checking in decreasing order each value $q \in \pi^*[q']$ until it stops at some $q < q'$. Thus, P_q is the longest prefix of $P_{q'}$ for which $P[q+1] = T[i]$, so that when the **while** loop terminates, $q + 1 = \sigma(P_{q'}T[i])$. Since $q' = \sigma(T_{i-1})$, Lemma 32.3 implies that $\sigma(T_{i-1}T[i]) = \sigma(P_{q'}T[i])$. Thus, we have

$$\begin{aligned}
 q + 1 &= \sigma(P_{q'}T[i]) \\
 &= \sigma(T_{i-1}T[i]) \\
 &= \sigma(T_i)
 \end{aligned}$$

when the **while** loop terminates. After line 9 increments q , we have $q = \sigma(T_i)$.

Line 12 is necessary in KMP-MATCHER, because otherwise, we might reference $P[m+1]$ on line 6 after finding an occurrence of P . (The argument that $q = \sigma(T_{i-1})$ upon the next execution of line 6 remains valid by the hint given in Exercise 32.4-8: $\delta(m, a) = \delta(\pi[m], a)$ or, equivalently, $\sigma(Pa) = \sigma(P_{\pi[m]}a)$ for any $a \in \Sigma$.) The remaining argument for the correctness of the Knuth-Morris-Pratt algorithm follows from the correctness of FINITE-AUTOMATON-MATCHER, since we have shown that KMP-MATCHER simulates the behavior of FINITE-AUTOMATON-MATCHER.

Exercises

32.4-1

Compute the prefix function π for the pattern ababbabbabbababbabb.

32.4-2

Give an upper bound on the size of $\pi^*[q]$ as a function of q . Give an example to show that your bound is tight.

32.4-3

Explain how to determine the occurrences of pattern P in the text T by examining the π function for the string PT (the string of length $m+n$ that is the concatenation of P and T).

32.4-4

Use an aggregate analysis to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-5

Use a potential function to show that the running time of KMP-MATCHER is $\Theta(n)$.

32.4-6

Show how to improve KMP-MATCHER by replacing the occurrence of π in line 7 (but not line 12) by π' , where π' is defined recursively for $q = 1, 2, \dots, m - 1$ by the equation

$$\pi'[q] = \begin{cases} 0 & \text{if } \pi[q] = 0, \\ \pi'[\pi[q]] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] = P[q + 1], \\ \pi[q] & \text{if } \pi[q] \neq 0 \text{ and } P[\pi[q] + 1] \neq P[q + 1]. \end{cases}$$

Explain why the modified algorithm is correct, and explain in what sense this change constitutes an improvement.

32.4-7

Give a linear-time algorithm to determine whether a text T is a cyclic rotation of another string T' . For example, `arc` and `car` are cyclic rotations of each other.

32.4-8 ★

Give an $O(m|\Sigma|)$ -time algorithm for computing the transition function δ for the string-matching automaton corresponding to a given pattern P . (*Hint:* Prove that $\delta(q, a) = \delta(\pi[q], a)$ if $q = m$ or $P[q + 1] \neq a$.)

Problems
32-1 String matching based on repetition factors

Let y^i denote the concatenation of string y with itself i times. For example, $(ab)^3 = ababab$. We say that a string $x \in \Sigma^*$ has **repetition factor** r if $x = y^r$ for some string $y \in \Sigma^*$ and some $r > 0$. Let $\rho(x)$ denote the largest r such that x has repetition factor r .

- a.** Give an efficient algorithm that takes as input a pattern $P[1..m]$ and computes the value $\rho(P_i)$ for $i = 1, 2, \dots, m$. What is the running time of your algorithm?

- b.** For any pattern $P[1..m]$, let $\rho^*(P)$ be defined as $\max_{1 \leq i \leq m} \rho(P_i)$. Prove that if the pattern P is chosen randomly from the set of all binary strings of length m , then the expected value of $\rho^*(P)$ is $O(1)$.
- c.** Argue that the following string-matching algorithm correctly finds all occurrences of pattern P in a text $T[1..n]$ in time $O(\rho^*(P)n + m)$:

REPETITION-MATCHER(P, T)

```

1   $m = P.length$ 
2   $n = T.length$ 
3   $k = 1 + \rho^*(P)$ 
4   $q = 0$ 
5   $s = 0$ 
6  while  $s \leq n - m$ 
7      if  $T[s + q + 1] == P[q + 1]$ 
8           $q = q + 1$ 
9          if  $q == m$ 
10             print "Pattern occurs with shift"  $s$ 
11      if  $q == m$  or  $T[s + q + 1] \neq P[q + 1]$ 
12           $s = s + \max(1, \lceil q/k \rceil)$ 
13       $q = 0$ 
```

This algorithm is due to Galil and Seiferas. By extending these ideas greatly, they obtained a linear-time string-matching algorithm that uses only $O(1)$ storage beyond what is required for P and T .

Chapter notes

The relation of string matching to the theory of finite automata is discussed by Aho, Hopcroft, and Ullman [5]. The Knuth-Morris-Pratt algorithm [214] was invented independently by Knuth and Pratt and by Morris; they published their work jointly. Reingold, Urban, and Gries [294] give an alternative treatment of the Knuth-Morris-Pratt algorithm. The Rabin-Karp algorithm was proposed by Karp and Rabin [201]. Galil and Seiferas [126] give an interesting deterministic linear-time string-matching algorithm that uses only $O(1)$ space beyond that required to store the pattern and text.

Computational geometry is the branch of computer science that studies algorithms for solving geometric problems. In modern engineering and mathematics, computational geometry has applications in such diverse fields as computer graphics, robotics, VLSI design, computer-aided design, molecular modeling, metallurgy, manufacturing, textile layout, forestry, and statistics. The input to a computational-geometry problem is typically a description of a set of geometric objects, such as a set of points, a set of line segments, or the vertices of a polygon in counterclockwise order. The output is often a response to a query about the objects, such as whether any of the lines intersect, or perhaps a new geometric object, such as the convex hull (smallest enclosing convex polygon) of the set of points.

In this chapter, we look at a few computational-geometry algorithms in two dimensions, that is, in the plane. We represent each input object by a set of points $\{p_1, p_2, p_3, \dots\}$, where each $p_i = (x_i, y_i)$ and $x_i, y_i \in \mathbb{R}$. For example, we represent an n -vertex polygon P by a sequence $\langle p_0, p_1, p_2, \dots, p_{n-1} \rangle$ of its vertices in order of their appearance on the boundary of P . Computational geometry can also apply to three dimensions, and even higher-dimensional spaces, but such problems and their solutions can be very difficult to visualize. Even in two dimensions, however, we can see a good sample of computational-geometry techniques.

Section 33.1 shows how to answer basic questions about line segments efficiently and accurately: whether one segment is clockwise or counterclockwise from another that shares an endpoint, which way we turn when traversing two adjoining line segments, and whether two line segments intersect. Section 33.2 presents a technique called “sweeping” that we use to develop an $O(n \lg n)$ -time algorithm for determining whether a set of n line segments contains any intersections. Section 33.3 gives two “rotational-sweep” algorithms that compute the convex hull (smallest enclosing convex polygon) of a set of n points: Graham’s scan, which runs in time $O(n \lg n)$, and Jarvis’s march, which takes $O(nh)$ time, where h is the number of vertices of the convex hull. Finally, Section 33.4 gives

an $O(n \lg n)$ -time divide-and-conquer algorithm for finding the closest pair of points in a set of n points in the plane.

33.1 Line-segment properties

Several of the computational-geometry algorithms in this chapter require answers to questions about the properties of line segments. A **convex combination** of two distinct points $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$ is any point $p_3 = (x_3, y_3)$ such that for some α in the range $0 \leq \alpha \leq 1$, we have $x_3 = \alpha x_1 + (1 - \alpha)x_2$ and $y_3 = \alpha y_1 + (1 - \alpha)y_2$. We also write that $p_3 = \alpha p_1 + (1 - \alpha)p_2$. Intuitively, p_3 is any point that is on the line passing through p_1 and p_2 and is on or between p_1 and p_2 on the line. Given two distinct points p_1 and p_2 , the **line segment** $\overline{p_1 p_2}$ is the set of convex combinations of p_1 and p_2 . We call p_1 and p_2 the **endpoints** of segment $\overline{p_1 p_2}$. Sometimes the ordering of p_1 and p_2 matters, and we speak of the **directed segment** $\overrightarrow{p_1 p_2}$. If p_1 is the **origin** $(0, 0)$, then we can treat the directed segment $\overrightarrow{p_1 p_2}$ as the **vector** p_2 .

In this section, we shall explore the following questions:

1. Given two directed segments $\overrightarrow{p_0 p_1}$ and $\overrightarrow{p_0 p_2}$, is $\overrightarrow{p_0 p_1}$ clockwise from $\overrightarrow{p_0 p_2}$ with respect to their common endpoint p_0 ?
2. Given two line segments $\overline{p_0 p_1}$ and $\overline{p_1 p_2}$, if we traverse $\overline{p_0 p_1}$ and then $\overline{p_1 p_2}$, do we make a left turn at point p_1 ?
3. Do line segments $\overline{p_1 p_2}$ and $\overline{p_3 p_4}$ intersect?

There are no restrictions on the given points.

We can answer each question in $O(1)$ time, which should come as no surprise since the input size of each question is $O(1)$. Moreover, our methods use only additions, subtractions, multiplications, and comparisons. We need neither division nor trigonometric functions, both of which can be computationally expensive and prone to problems with round-off error. For example, the “straightforward” method of determining whether two segments intersect—compute the line equation of the form $y = mx + b$ for each segment (m is the slope and b is the y -intercept), find the point of intersection of the lines, and check whether this point is on both segments—uses division to find the point of intersection. When the segments are nearly parallel, this method is very sensitive to the precision of the division operation on real computers. The method in this section, which avoids division, is much more accurate.

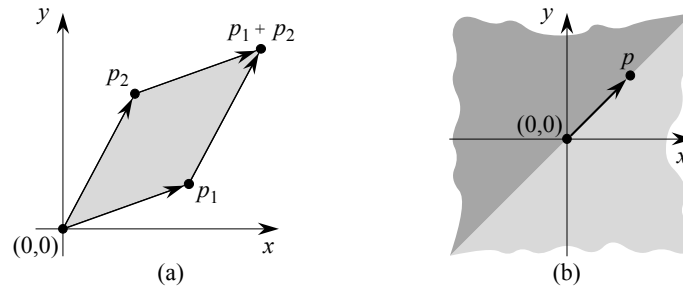


Figure 33.1 (a) The cross product of vectors p_1 and p_2 is the signed area of the parallelogram. (b) The lightly shaded region contains vectors that are clockwise from p . The darkly shaded region contains vectors that are counterclockwise from p .

Cross products

Computing cross products lies at the heart of our line-segment methods. Consider vectors p_1 and p_2 , shown in Figure 33.1(a). We can interpret the **cross product** $p_1 \times p_2$ as the signed area of the parallelogram formed by the points $(0, 0)$, p_1 , p_2 , and $p_1 + p_2 = (x_1 + x_2, y_1 + y_2)$. An equivalent, but more useful, definition gives the cross product as the determinant of a matrix:¹

$$\begin{aligned} p_1 \times p_2 &= \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} \\ &= x_1 y_2 - x_2 y_1 \\ &= -p_2 \times p_1. \end{aligned}$$

If $p_1 \times p_2$ is positive, then p_1 is clockwise from p_2 with respect to the origin $(0, 0)$; if this cross product is negative, then p_1 is counterclockwise from p_2 . (See Exercise 33.1-1.) Figure 33.1(b) shows the clockwise and counterclockwise regions relative to a vector p . A boundary condition arises if the cross product is 0; in this case, the vectors are **colinear**, pointing in either the same or opposite directions.

To determine whether a directed segment $\overrightarrow{p_0 p_1}$ is closer to a directed segment $\overrightarrow{p_0 p_2}$ in a clockwise direction or in a counterclockwise direction with respect to their common endpoint p_0 , we simply translate to use p_0 as the origin. That is, we let $p_1 - p_0$ denote the vector $p'_1 = (x'_1, y'_1)$, where $x'_1 = x_1 - x_0$ and $y'_1 = y_1 - y_0$, and we define $p_2 - p_0$ similarly. We then compute the cross product

¹Actually, the cross product is a three-dimensional concept. It is a vector that is perpendicular to both p_1 and p_2 according to the “right-hand rule” and whose magnitude is $|x_1 y_2 - x_2 y_1|$. In this chapter, however, we find it convenient to treat the cross product simply as the value $x_1 y_2 - x_2 y_1$.

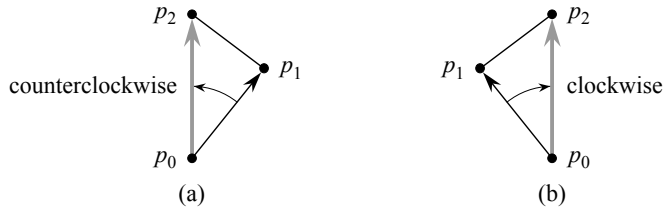


Figure 33.2 Using the cross product to determine how consecutive line segments $\overline{p_0p_1}$ and $\overline{p_1p_2}$ turn at point p_1 . We check whether the directed segment $\overrightarrow{p_0p_2}$ is clockwise or counterclockwise relative to the directed segment $\overrightarrow{p_0p_1}$. **(a)** If counterclockwise, the points make a left turn. **(b)** If clockwise, they make a right turn.

$$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0).$$

If this cross product is positive, then $\overrightarrow{p_0p_1}$ is clockwise from $\overrightarrow{p_0p_2}$; if negative, it is counterclockwise.

Determining whether consecutive segments turn left or right

Our next question is whether two consecutive line segments $\overline{p_0p_1}$ and $\overline{p_1p_2}$ turn left or right at point p_1 . Equivalently, we want a method to determine which way a given angle $\angle p_0p_1p_2$ turns. Cross products allow us to answer this question without computing the angle. As Figure 33.2 shows, we simply check whether directed segment $\overrightarrow{p_0p_2}$ is clockwise or counterclockwise relative to directed segment $\overrightarrow{p_0p_1}$. To do so, we compute the cross product $(p_2 - p_0) \times (p_1 - p_0)$. If the sign of this cross product is negative, then $\overrightarrow{p_0p_2}$ is counterclockwise with respect to $\overrightarrow{p_0p_1}$, and thus we make a left turn at p_1 . A positive cross product indicates a clockwise orientation and a right turn. A cross product of 0 means that points p_0 , p_1 , and p_2 are colinear.

Determining whether two line segments intersect

To determine whether two line segments intersect, we check whether each segment straddles the line containing the other. A segment $\overline{p_1p_2}$ **straddles** a line if point p_1 lies on one side of the line and point p_2 lies on the other side. A boundary case arises if p_1 or p_2 lies directly on the line. Two line segments intersect if and only if either (or both) of the following conditions holds:

1. Each segment straddles the line containing the other.
2. An endpoint of one segment lies on the other segment. (This condition comes from the boundary case.)

The following procedures implement this idea. SEGMENTS-INTERSECT returns TRUE if segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ intersect and FALSE if they do not. It calls the subroutines DIRECTION, which computes relative orientations using the cross-product method above, and ON-SEGMENT, which determines whether a point known to be colinear with a segment lies on that segment.

SEGMENTS-INTERSECT(p_1, p_2, p_3, p_4)

```

1   $d_1 = \text{DIRECTION}(p_3, p_4, p_1)$ 
2   $d_2 = \text{DIRECTION}(p_3, p_4, p_2)$ 
3   $d_3 = \text{DIRECTION}(p_1, p_2, p_3)$ 
4   $d_4 = \text{DIRECTION}(p_1, p_2, p_4)$ 
5  if  $((d_1 > 0 \text{ and } d_2 < 0) \text{ or } (d_1 < 0 \text{ and } d_2 > 0)) \text{ and}$ 
    $((d_3 > 0 \text{ and } d_4 < 0) \text{ or } (d_3 < 0 \text{ and } d_4 > 0))$ 
6    return TRUE
7  elseif  $d_1 == 0 \text{ and } \text{ON-SEGMENT}(p_3, p_4, p_1)$ 
8    return TRUE
9  elseif  $d_2 == 0 \text{ and } \text{ON-SEGMENT}(p_3, p_4, p_2)$ 
10   return TRUE
11 elseif  $d_3 == 0 \text{ and } \text{ON-SEGMENT}(p_1, p_2, p_3)$ 
12   return TRUE
13 elseif  $d_4 == 0 \text{ and } \text{ON-SEGMENT}(p_1, p_2, p_4)$ 
14   return TRUE
15 else return FALSE
```

DIRECTION(p_i, p_j, p_k)

```

1  return  $(p_k - p_i) \times (p_j - p_i)$ 
```

ON-SEGMENT(p_i, p_j, p_k)

```

1  if  $\min(x_i, x_j) \leq x_k \leq \max(x_i, x_j) \text{ and } \min(y_i, y_j) \leq y_k \leq \max(y_i, y_j)$ 
2    return TRUE
3  else return FALSE
```

SEGMENTS-INTERSECT works as follows. Lines 1–4 compute the relative orientation d_i of each endpoint p_i with respect to the other segment. If all the relative orientations are nonzero, then we can easily determine whether segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ intersect, as follows. Segment $\overline{p_1p_2}$ straddles the line containing segment $\overline{p_3p_4}$ if directed segments $\overrightarrow{p_3p_1}$ and $\overrightarrow{p_3p_2}$ have opposite orientations relative to $\overrightarrow{p_3p_4}$. In this case, the signs of d_1 and d_2 differ. Similarly, $\overline{p_3p_4}$ straddles the line containing $\overline{p_1p_2}$ if the signs of d_3 and d_4 differ. If the test of line 5 is true, then the segments straddle each other, and SEGMENTS-INTERSECT returns TRUE. Figure 33.3(a) shows this case. Otherwise, the segments do not straddle

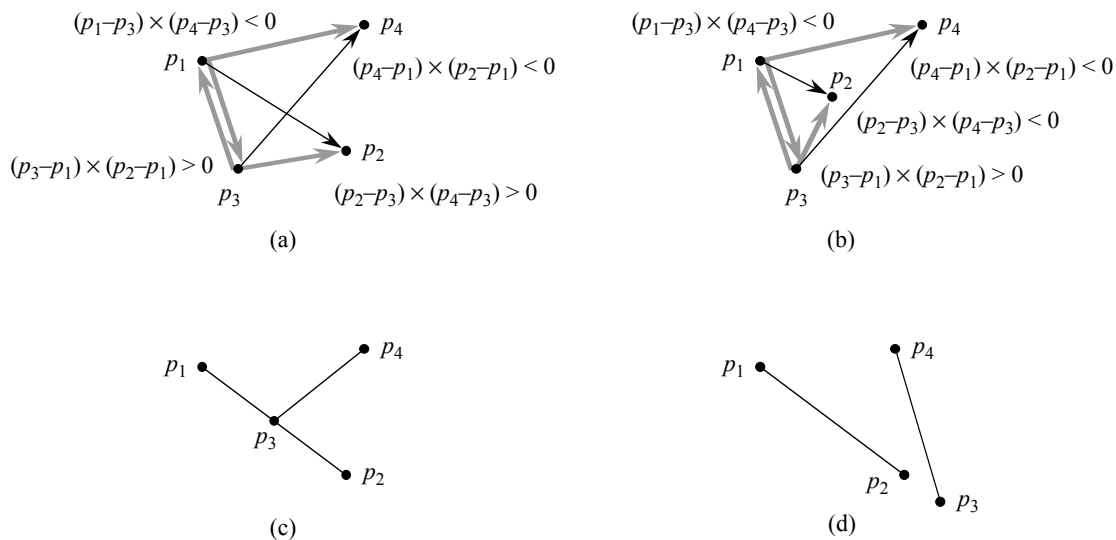


Figure 33.3 Cases in the procedure SEGMENTS-INTERSECT. **(a)** The segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ straddle each other's lines. Because $\overline{p_3p_4}$ straddles the line containing $\overline{p_1p_2}$, the signs of the cross products $(p_3 - p_1) \times (p_2 - p_1)$ and $(p_4 - p_1) \times (p_2 - p_1)$ differ. Because $\overline{p_1p_2}$ straddles the line containing $\overline{p_3p_4}$, the signs of the cross products $(p_1 - p_3) \times (p_4 - p_3)$ and $(p_2 - p_3) \times (p_4 - p_3)$ differ. **(b)** Segment $\overline{p_3p_4}$ straddles the line containing $\overline{p_1p_2}$, but $\overline{p_1p_2}$ does not straddle the line containing $\overline{p_3p_4}$. The signs of the cross products $(p_1 - p_3) \times (p_4 - p_3)$ and $(p_2 - p_3) \times (p_4 - p_3)$ are the same. **(c)** Point p_3 is colinear with $\overline{p_1p_2}$ and is between p_1 and p_2 . **(d)** Point p_3 is colinear with $\overline{p_1p_2}$, but it is not between p_1 and p_2 . The segments do not intersect.

each other's lines, although a boundary case may apply. If all the relative orientations are nonzero, no boundary case applies. All the tests against 0 in lines 7–13 then fail, and SEGMENTS-INTERSECT returns FALSE in line 15. Figure 33.3(b) shows this case.

A boundary case occurs if any relative orientation d_k is 0. Here, we know that p_k is colinear with the other segment. It is directly on the other segment if and only if it is between the endpoints of the other segment. The procedure ON-SEGMENT returns whether p_k is between the endpoints of segment $\overline{p_i p_j}$, which will be the other segment when called in lines 7–13; the procedure assumes that p_k is colinear with segment $\overline{p_i p_j}$. Figures 33.3(c) and (d) show cases with colinear points. In Figure 33.3(c), p_3 is on $\overline{p_1p_2}$, and so SEGMENTS-INTERSECT returns TRUE in line 12. No endpoints are on other segments in Figure 33.3(d), and so SEGMENTS-INTERSECT returns FALSE in line 15.

Other applications of cross products

Later sections of this chapter introduce additional uses for cross products. In Section 33.3, we shall need to sort a set of points according to their polar angles with respect to a given origin. As Exercise 33.1-3 asks you to show, we can use cross products to perform the comparisons in the sorting procedure. In Section 33.2, we shall use red-black trees to maintain the vertical ordering of a set of line segments. Rather than keeping explicit key values which we compare to each other in the red-black tree code, we shall compute a cross-product to determine which of two segments that intersect a given vertical line is above the other.

Exercises

33.1-1

Prove that if $p_1 \times p_2$ is positive, then vector p_1 is clockwise from vector p_2 with respect to the origin $(0, 0)$ and that if this cross product is negative, then p_1 is counterclockwise from p_2 .

33.1-2

Professor van Pelt proposes that only the x -dimension needs to be tested in line 1 of ON-SEGMENT. Show why the professor is wrong.

33.1-3

The **polar angle** of a point p_1 with respect to an origin point p_0 is the angle of the vector $p_1 - p_0$ in the usual polar coordinate system. For example, the polar angle of $(3, 5)$ with respect to $(2, 4)$ is the angle of the vector $(1, 1)$, which is 45 degrees or $\pi/4$ radians. The polar angle of $(3, 3)$ with respect to $(2, 4)$ is the angle of the vector $(1, -1)$, which is 315 degrees or $7\pi/4$ radians. Write pseudocode to sort a sequence $\langle p_1, p_2, \dots, p_n \rangle$ of n points according to their polar angles with respect to a given origin point p_0 . Your procedure should take $O(n \lg n)$ time and use cross products to compare angles.

33.1-4

Show how to determine in $O(n^2 \lg n)$ time whether any three points in a set of n points are colinear.

33.1-5

A **polygon** is a piecewise-linear, closed curve in the plane. That is, it is a curve ending on itself that is formed by a sequence of straight-line segments, called the **sides** of the polygon. A point joining two consecutive sides is a **vertex** of the polygon. If the polygon is **simple**, as we shall generally assume, it does not cross itself. The set of points in the plane enclosed by a simple polygon forms the **interior** of

the polygon, the set of points on the polygon itself forms its **boundary**, and the set of points surrounding the polygon forms its **exterior**. A simple polygon is **convex** if, given any two points on its boundary or in its interior, all points on the line segment drawn between them are contained in the polygon's boundary or interior. A vertex of a convex polygon cannot be expressed as a convex combination of any two distinct points on the boundary or in the interior of the polygon.

Professor Amundsen proposes the following method to determine whether a sequence $\langle p_0, p_1, \dots, p_{n-1} \rangle$ of n points forms the consecutive vertices of a convex polygon. Output “yes” if the set $\{\angle p_i p_{i+1} p_{i+2} : i = 0, 1, \dots, n-1\}$, where subscript addition is performed modulo n , does not contain both left turns and right turns; otherwise, output “no.” Show that although this method runs in linear time, it does not always produce the correct answer. Modify the professor's method so that it always produces the correct answer in linear time.

33.1-6

Given a point $p_0 = (x_0, y_0)$, the **right horizontal ray** from p_0 is the set of points $\{p_i = (x_i, y_i) : x_i \geq x_0 \text{ and } y_i = y_0\}$, that is, it is the set of points due right of p_0 along with p_0 itself. Show how to determine whether a given right horizontal ray from p_0 intersects a line segment $\overline{p_1 p_2}$ in $O(1)$ time by reducing the problem to that of determining whether two line segments intersect.

33.1-7

One way to determine whether a point p_0 is in the interior of a simple, but not necessarily convex, polygon P is to look at any ray from p_0 and check that the ray intersects the boundary of P an odd number of times but that p_0 itself is not on the boundary of P . Show how to compute in $\Theta(n)$ time whether a point p_0 is in the interior of an n -vertex polygon P . (*Hint:* Use Exercise 33.1-6. Make sure your algorithm is correct when the ray intersects the polygon boundary at a vertex and when the ray overlaps a side of the polygon.)

33.1-8

Show how to compute the area of an n -vertex simple, but not necessarily convex, polygon in $\Theta(n)$ time. (See Exercise 33.1-5 for definitions pertaining to polygons.)

33.2 Determining whether any pair of segments intersects

This section presents an algorithm for determining whether any two line segments in a set of segments intersect. The algorithm uses a technique known as “sweeping,” which is common to many computational-geometry algorithms. Moreover, as

the exercises at the end of this section show, this algorithm, or simple variations of it, can help solve other computational-geometry problems.

The algorithm runs in $O(n \lg n)$ time, where n is the number of segments we are given. It determines only whether or not any intersection exists; it does not print all the intersections. (By Exercise 33.2-1, it takes $\Omega(n^2)$ time in the worst case to find *all* the intersections in a set of n line segments.)

In *sweeping*, an imaginary vertical *sweep line* passes through the given set of geometric objects, usually from left to right. We treat the spatial dimension that the sweep line moves across, in this case the x -dimension, as a dimension of time. Sweeping provides a method for ordering geometric objects, usually by placing them into a dynamic data structure, and for taking advantage of relationships among them. The line-segment-intersection algorithm in this section considers all the line-segment endpoints in left-to-right order and checks for an intersection each time it encounters an endpoint.

To describe and prove correct our algorithm for determining whether any two of n line segments intersect, we shall make two simplifying assumptions. First, we assume that no input segment is vertical. Second, we assume that no three input segments intersect at a single point. Exercises 33.2-8 and 33.2-9 ask you to show that the algorithm is robust enough that it needs only a slight modification to work even when these assumptions do not hold. Indeed, removing such simplifying assumptions and dealing with boundary conditions often present the most difficult challenges when programming computational-geometry algorithms and proving their correctness.

Ordering segments

Because we assume that there are no vertical segments, we know that any input segment intersecting a given vertical sweep line intersects it at a single point. Thus, we can order the segments that intersect a vertical sweep line according to the y -coordinates of the points of intersection.

To be more precise, consider two segments s_1 and s_2 . We say that these segments are *comparable* at x if the vertical sweep line with x -coordinate x intersects both of them. We say that s_1 is *above* s_2 at x , written $s_1 \succ_x s_2$, if s_1 and s_2 are comparable at x and the intersection of s_1 with the sweep line at x is higher than the intersection of s_2 with the same sweep line, or if s_1 and s_2 intersect at the sweep line. In Figure 33.4(a), for example, we have the relationships $a \succ_r c$, $a \succ_t b$, $b \succ_t c$, $a \succ_t c$, and $b \succ_u c$. Segment d is not comparable with any other segment.

For any given x , the relation “ $\succ_x$ ” is a total preorder (see Section B.2) for all segments that intersect the sweep line at x . That is, the relation is transitive, and if segments s_1 and s_2 each intersect the sweep line at x , then either $s_1 \succ_x s_2$ or $s_2 \succ_x s_1$, or both (if s_1 and s_2 intersect at the sweep line). (The relation $\succ_x$ is

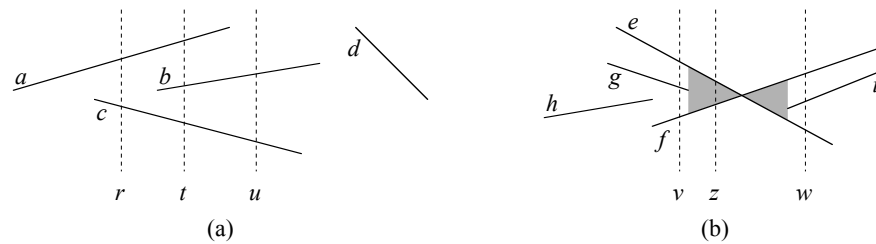


Figure 33.4 The ordering among line segments at various vertical sweep lines. **(a)** We have $a \succ_r c$, $a \succ_t b$, $b \succ_t c$, $a \succ_t c$, and $b \succ_u c$. Segment d is comparable with no other segment shown. **(b)** When segments e and f intersect, they reverse their orders: we have $e \succ_v f$ but $f \succ_w e$. Any sweep line (such as z) that passes through the shaded region has e and f consecutive in the ordering given by the relation $\succ_z$.

also reflexive, but neither symmetric nor antisymmetric.) The total preorder may differ for differing values of x , however, as segments enter and leave the ordering. A segment enters the ordering when its left endpoint is encountered by the sweep, and it leaves the ordering when its right endpoint is encountered.

What happens when the sweep line passes through the intersection of two segments? As Figure 33.4(b) shows, the segments reverse their positions in the total preorder. Sweep lines v and w are to the left and right, respectively, of the point of intersection of segments e and f , and we have $e \succ_v f$ and $f \succ_w e$. Note that because we assume that no three segments intersect at the same point, there must be some vertical sweep line x for which intersecting segments e and f are *consecutive* in the total preorder $\succ_x$. Any sweep line that passes through the shaded region of Figure 33.4(b), such as z , has e and f consecutive in its total preorder.

Moving the sweep line

Sweeping algorithms typically manage two sets of data:

1. The **sweep-line status** gives the relationships among the objects that the sweep line intersects.
2. The **event-point schedule** is a sequence of points, called **event points**, which we order from left to right according to their x -coordinates. As the sweep progresses from left to right, whenever the sweep line reaches the x -coordinate of an event point, the sweep halts, processes the event point, and then resumes. Changes to the sweep-line status occur only at event points.

For some algorithms (the algorithm asked for in Exercise 33.2-7, for example), the event-point schedule develops dynamically as the algorithm progresses. The algorithm at hand, however, determines all the event points before the sweep, based

solely on simple properties of the input data. In particular, each segment endpoint is an event point. We sort the segment endpoints by increasing x -coordinate and proceed from left to right. (If two or more endpoints are *covertical*, i.e., they have the same x -coordinate, we break the tie by putting all the covertical left endpoints before the covertical right endpoints. Within a set of covertical left endpoints, we put those with lower y -coordinates first, and we do the same within a set of covertical right endpoints.) When we encounter a segment's left endpoint, we insert the segment into the sweep-line status, and we delete the segment from the sweep-line status upon encountering its right endpoint. Whenever two segments first become consecutive in the total preorder, we check whether they intersect.

The sweep-line status is a total preorder T , for which we require the following operations:

- $\text{INSERT}(T, s)$: insert segment s into T .
- $\text{DELETE}(T, s)$: delete segment s from T .
- $\text{ABOVE}(T, s)$: return the segment immediately above segment s in T .
- $\text{BELOW}(T, s)$: return the segment immediately below segment s in T .

It is possible for segments s_1 and s_2 to be mutually above each other in the total preorder T ; this situation can occur if s_1 and s_2 intersect at the sweep line whose total preorder is given by T . In this case, the two segments may appear in either order in T .

If the input contains n segments, we can perform each of the operations INSERT, DELETE, ABOVE, and BELOW in $O(\lg n)$ time using red-black trees. Recall that the red-black-tree operations in Chapter 13 involve comparing keys. We can replace the key comparisons by comparisons that use cross products to determine the relative ordering of two segments (see Exercise 33.2-2).

Segment-intersection pseudocode

The following algorithm takes as input a set S of n line segments, returning the boolean value TRUE if any pair of segments in S intersects, and FALSE otherwise. A red-black tree maintains the total preorder T .

ANY-SEGMENTS-INTERSECT(S)

```

1   $T = \emptyset$ 
2  sort the endpoints of the segments in  $S$  from left to right,
    breaking ties by putting left endpoints before right endpoints
    and breaking further ties by putting points with lower
     $y$ -coordinates first
3  for each point  $p$  in the sorted list of endpoints
4      if  $p$  is the left endpoint of a segment  $s$ 
5          INSERT( $T, s$ )
6          if (ABOVE( $T, s$ ) exists and intersects  $s$ )
              or (BELOW( $T, s$ ) exists and intersects  $s$ )
7              return TRUE
8      if  $p$  is the right endpoint of a segment  $s$ 
9          if both ABOVE( $T, s$ ) and BELOW( $T, s$ ) exist
              and ABOVE( $T, s$ ) intersects BELOW( $T, s$ )
10             return TRUE
11             DELETE( $T, s$ )
12 return FALSE

```

Figure 33.5 illustrates how the algorithm works. Line 1 initializes the total preorder to be empty. Line 2 determines the event-point schedule by sorting the $2n$ segment endpoints from left to right, breaking ties as described above. One way to perform line 2 is by lexicographically sorting the endpoints on (x, e, y) , where x and y are the usual coordinates, $e = 0$ for a left endpoint, and $e = 1$ for a right endpoint.

Each iteration of the **for** loop of lines 3–11 processes one event point p . If p is the left endpoint of a segment s , line 5 adds s to the total preorder, and lines 6–7 return TRUE if s intersects either of the segments it is consecutive with in the total preorder defined by the sweep line passing through p . (A boundary condition occurs if p lies on another segment s' . In this case, we require only that s and s' be placed consecutively into T .) If p is the right endpoint of a segment s , then we need to delete s from the total preorder. But first, lines 9–10 return TRUE if there is an intersection between the segments surrounding s in the total preorder defined by the sweep line passing through p . If these segments do not intersect, line 11 deletes segment s from the total preorder. If the segments surrounding segment s intersect, they would have become consecutive after deleting s had the **return** statement in line 10 not prevented line 11 from executing. The correctness argument, which follows, will make it clear why it suffices to check the segments surrounding s . Finally, if we never find any intersections after having processed all $2n$ event points, line 12 returns FALSE.

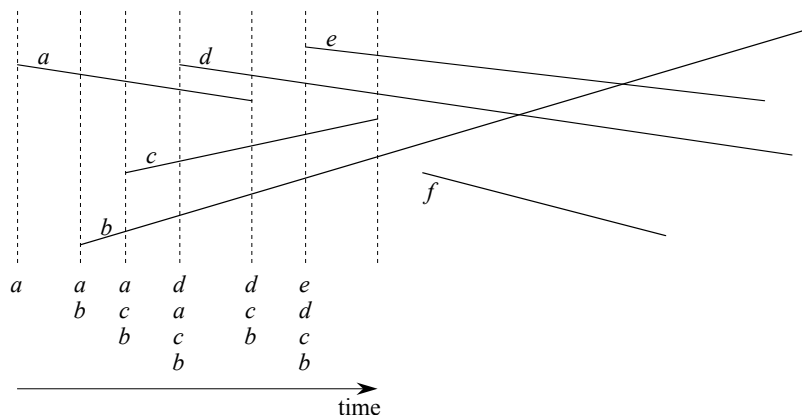


Figure 33.5 The execution of ANY-SEGMENTS-INTERSECT. Each dashed line is the sweep line at an event point. Except for the rightmost sweep line, the ordering of segment names below each sweep line corresponds to the total preorder T at the end of the **for** loop processing the corresponding event point. The rightmost sweep line occurs when processing the right endpoint of segment c ; because segments d and b surround c and intersect each other, the procedure returns TRUE.

Correctness

To show that ANY-SEGMENTS-INTERSECT is correct, we will prove that the call ANY-SEGMENTS-INTERSECT(S) returns TRUE if and only if there is an intersection among the segments in S .

It is easy to see that ANY-SEGMENTS-INTERSECT returns TRUE (on lines 7 and 10) only if it finds an intersection between two of the input segments. Hence, if it returns TRUE, there is an intersection.

We also need to show the converse: that if there is an intersection, then ANY-SEGMENTS-INTERSECT returns TRUE. Let us suppose that there is at least one intersection. Let p be the leftmost intersection point, breaking ties by choosing the point with the lowest y -coordinate, and let a and b be the segments that intersect at p . Since no intersections occur to the left of p , the order given by T is correct at all points to the left of p . Because no three segments intersect at the same point, a and b become consecutive in the total preorder at some sweep line z .² Moreover, z is to the left of p or goes through p . Some segment endpoint q on sweep line z

²If we allow three segments to intersect at the same point, there may be an intervening segment c that intersects both a and b at point p . That is, we may have $a \succ_w c$ and $c \succ_w b$ for all sweep lines w to the left of p for which $a \succ_w b$. Exercise 33.2-8 asks you to show that ANY-SEGMENTS-INTERSECT is correct even if three segments do intersect at the same point.

is the event point at which a and b become consecutive in the total preorder. If p is on sweep line z , then $q = p$. If p is not on sweep line z , then q is to the left of p . In either case, the order given by T is correct just before encountering q . (Here is where we use the lexicographic order in which the algorithm processes event points. Because p is the lowest of the leftmost intersection points, even if p is on sweep line z and some other intersection point p' is on z , event point $q = p$ is processed before the other intersection p' can interfere with the total preorder T . Moreover, even if p is the left endpoint of one segment, say a , and the right endpoint of the other segment, say b , because left endpoint events occur before right endpoint events, segment b is in T upon first encountering segment a .) Either event point q is processed by ANY-SEGMENTS-INTERSECT or it is not processed.

If q is processed by ANY-SEGMENTS-INTERSECT, only two possible actions may occur:

1. Either a or b is inserted into T , and the other segment is above or below it in the total preorder. Lines 4–7 detect this case.
2. Segments a and b are already in T , and a segment between them in the total preorder is deleted, making a and b become consecutive. Lines 8–11 detect this case.

In either case, we find the intersection p and ANY-SEGMENTS-INTERSECT returns TRUE.

If event point q is not processed by ANY-SEGMENTS-INTERSECT, the procedure must have returned before processing all event points. This situation could have occurred only if ANY-SEGMENTS-INTERSECT had already found an intersection and returned TRUE.

Thus, if there is an intersection, ANY-SEGMENTS-INTERSECT returns TRUE. As we have already seen, if ANY-SEGMENTS-INTERSECT returns TRUE, there is an intersection. Therefore, ANY-SEGMENTS-INTERSECT always returns a correct answer.

Running time

If set S contains n segments, then ANY-SEGMENTS-INTERSECT runs in time $O(n \lg n)$. Line 1 takes $O(1)$ time. Line 2 takes $O(n \lg n)$ time, using merge sort or heapsort. The **for** loop of lines 3–11 iterates at most once per event point, and so with $2n$ event points, the loop iterates at most $2n$ times. Each iteration takes $O(\lg n)$ time, since each red-black-tree operation takes $O(\lg n)$ time and, using the method of Section 33.1, each intersection test takes $O(1)$ time. The total time is thus $O(n \lg n)$.

Exercises

33.2-1

Show that a set of n line segments may contain $\Theta(n^2)$ intersections.

33.2-2

Given two segments a and b that are comparable at x , show how to determine in $O(1)$ time which of $a \succ_x b$ or $b \succ_x a$ holds. Assume that neither segment is vertical. (*Hint:* If a and b do not intersect, you can just use cross products. If a and b intersect—which you can of course determine using only cross products—you can still use only addition, subtraction, and multiplication, avoiding division. Of course, in the application of the $\succ_x$ relation used here, if a and b intersect, we can just stop and declare that we have found an intersection.)

33.2-3

Professor Mason suggests that we modify ANY-SEGMENTS-INTERSECT so that instead of returning upon finding an intersection, it prints the segments that intersect and continues on to the next iteration of the **for** loop. The professor calls the resulting procedure PRINT-INTERSECTING-SEGMENTS and claims that it prints all intersections, from left to right, as they occur in the set of line segments. Professor Dixon disagrees, claiming that Professor Mason's idea is incorrect. Which professor is right? Will PRINT-INTERSECTING-SEGMENTS always find the leftmost intersection first? Will it always find all the intersections?

33.2-4

Give an $O(n \lg n)$ -time algorithm to determine whether an n -vertex polygon is simple.

33.2-5

Give an $O(n \lg n)$ -time algorithm to determine whether two simple polygons with a total of n vertices intersect.

33.2-6

A *disk* consists of a circle plus its interior and is represented by its center point and radius. Two disks intersect if they have any point in common. Give an $O(n \lg n)$ -time algorithm to determine whether any two disks in a set of n intersect.

33.2-7

Given a set of n line segments containing a total of k intersections, show how to output all k intersections in $O((n + k) \lg n)$ time.

33.2-8

Argue that ANY-SEGMENTS-INTERSECT works correctly even if three or more segments intersect at the same point.

33.2-9

Show that ANY-SEGMENTS-INTERSECT works correctly in the presence of vertical segments if we treat the bottom endpoint of a vertical segment as if it were a left endpoint and the top endpoint as if it were a right endpoint. How does your answer to Exercise 33.2-2 change if we allow vertical segments?

33.3 Finding the convex hull

The **convex hull** of a set Q of points, denoted by $\text{CH}(Q)$, is the smallest convex polygon P for which each point in Q is either on the boundary of P or in its interior. (See Exercise 33.1-5 for a precise definition of a convex polygon.) We implicitly assume that all points in the set Q are unique and that Q contains at least three points which are not colinear. Intuitively, we can think of each point in Q as being a nail sticking out from a board. The convex hull is then the shape formed by a tight rubber band that surrounds all the nails. Figure 33.6 shows a set of points and its convex hull.

In this section, we shall present two algorithms that compute the convex hull of a set of n points. Both algorithms output the vertices of the convex hull in counterclockwise order. The first, known as Graham's scan, runs in $O(n \lg n)$ time. The second, called Jarvis's march, runs in $O(nh)$ time, where h is the number of vertices of the convex hull. As Figure 33.6 illustrates, every vertex of $\text{CH}(Q)$ is a

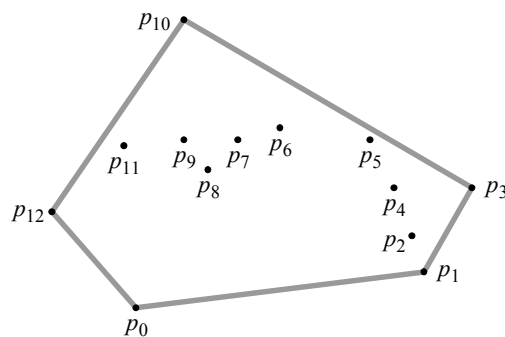


Figure 33.6 A set of points $Q = \{p_0, p_1, \dots, p_{12}\}$ with its convex hull $\text{CH}(Q)$ in gray.